

IEEE18 Intelligent Fault Analysis

A **Python-based intelligent power-system fault analysis platform** built around an 18-bus electrical network.

The project combines **power-system analysis, numerical computation, fault simulation, machine learning, and interactive visualization** into a single software-only platform.

 **No hardware required.** The entire project is developed and simulated using Python.

Project Objective

The goal of this project is to develop a complete software platform capable of:

- Performing power-system load-flow analysis
- Modeling an 18-bus electrical network
- Constructing the bus admittance matrix (Y-bus)
- Solving power flow using the Newton-Raphson method
- Performing symmetrical-component analysis
- Simulating different types of electrical faults
- Automatically generating fault scenarios
- Extracting electrical features from simulated faults
- Classifying faults using machine-learning models
- Estimating fault location
- Assessing fault severity
- Visualizing system conditions and fault behavior
- Providing an interactive engineering dashboard

Planned Architecture

flowchart TD

```
A["18-BUS POWER SYSTEM"] --> B["Y-BUS MATRIX"]
```

```
B --> C["NEWTON-RAPHSON<br/>LOAD FLOW"]
C --> D["SEQUENCE NETWORKS"]
D --> E["FAULT SIMULATION"]
```

```
E --> F["SLG<br/>Single Line-to-Ground"]
E --> G["LL<br/>Line-to-Line"]
E --> H["DLG<br/>Double Line-to-Ground"]
E --> I["3-PHASE<br/>Three-Phase Fault"]
```

```
F --> J["FEATURE EXTRACTION"]
G --> J
H --> J
I --> J
```

```
J --> K["MACHINE LEARNING"]
```

```
K --> L["FAULT TYPE"]
K --> M["FAULT LOCATION"]
K --> N["SEVERITY"]
```

```
L --> O["INTERACTIVE DASHBOARD"]
M --> O
N --> O
```

```
classDef highlightStyle fill:#f59e0b,stroke:#d97706,color:#000,font-weight:bold
classDef foundationStyle fill:#1e40af,stroke:#0c2340,color:#fff,font-weight:bold
classDef faultStyle fill:#ff8c00,stroke:#d97706,color:#fff,font-weight:bold
classDef processingStyle fill:#6b8e23,stroke:#556b2f,color:#fff,font-weight:bold
```

```
class A,O highlightStyle
class B,C,D,E foundationStyle
class F,G,H,I faultStyle
class J,K,L,M,N processingStyle
```

Development Status

Stage 1 — Power-System Foundation

- ☒ 18-bus system data
- ☒ Line/network data
- ☒ Generator data
- ☒ Y-bus construction
- ☒ Newton-Raphson load-flow solver
- ☒ Convergence checking
- ☒ Automated unit tests

Stage 2 — Fault Analysis

- ☐ Positive-sequence network
- ☐ Negative-sequence network
- ☐ Zero-sequence network
- ☐ Single Line-to-Ground (SLG) fault
- ☐ Line-to-Line (LL) fault
- ☐ Double Line-to-Ground (DLG) fault
- ☐ Three-phase fault
- ☐ Variable fault impedance
- ☐ Fault analysis at all buses

Stage 3 — Dataset Generation

- ☐ Automatic fault-scenario generation
- ☐ Variable fault locations
- ☐ Variable fault impedances
- ☐ Multiple operating conditions
- ☐ Feature extraction
- ☐ Dataset validation

Stage 4 — Machine Learning

- ☐ Fault/no-fault detection
- ☐ Fault-type classification

- ☐ Faulted-bus classification
- ☐ Fault-severity estimation
- ☐ Model comparison
- ☐ Model evaluation and explainability

Stage 5 — Visualization & Dashboard

- ☐ Bus-voltage visualization
- ☐ Fault-current visualization
- ☐ Three-phase waveforms
- ☐ Phasor visualization
- ☐ Network visualization
- ☐ Interactive Streamlit dashboard
- ☐ Automated fault reports

Project Structure

```
IEEE18-Intelligent-Fault-Analysis/  
|  
├─ data/  
|   ├── __init__.py  
|   └─ system_data.py  
|  
├─ power_system/  
|   ├── __init__.py  
|   ├── ybus.py  
|   ├── load_flow.py  
|   └─ main.py  
|  
├─ dataset/  
|   └─  
|  
├─ ml/  
|   └─  
|
```

```
├─ visualization/
│   └─
│
├─ dashboard/
│   └─
│
├─ tests/
│   ├── __init__.py
│   ├── test_ybus.py
│   └─ test_load_flow.py
│
├─ results/
│   └─
│
├─ requirements.txt
└─ README.md
```

Stage 1 — Power-System Analysis

The first stage implements the fundamental numerical power-system calculations directly in Python.

Y-Bus Construction

The bus admittance matrix is constructed from the transmission-line parameters using the network's π -model representation.

Y-BUS

$$\begin{bmatrix} Y_{11} & \dots & Y_{18} \\ \vdots & & \vdots \\ Y_{18} & \dots & Y_{18} \end{bmatrix}$$

Newton-Raphson Load Flow

The AC load-flow solver determines:

- Bus voltage magnitudes
- Bus voltage angles
- Active-power injections
- Reactive-power injections
- Convergence status
- Iteration count
- Maximum power mismatch

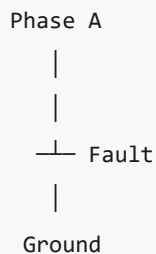
The solver supports:

- Slack bus
- PV buses
- PQ buses

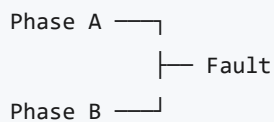
Planned Fault Analysis

The fault-analysis engine will support the major symmetrical-component fault types:

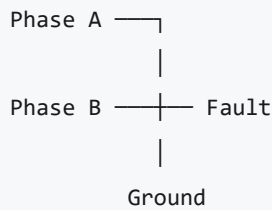
Single Line-to-Ground



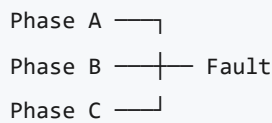
Line-to-Line



Double Line-to-Ground



Three-Phase Fault



Fault calculations will use the appropriate sequence networks and bus-dependent sequence impedances.



Machine Learning

After the electrical simulation engine is validated, simulated fault cases will be automatically converted into a machine-learning dataset.

Potential features include:

- Three-phase voltages
- Three-phase currents
- Positive-sequence quantities
- Negative-sequence quantities
- Zero-sequence quantities
- RMS voltage
- RMS current
- Voltage imbalance
- Current imbalance
- Fault current
- Fault impedance
- Power-system operating conditions

Potential models:

- Logistic Regression
- Decision Tree
- Random Forest
- Support Vector Machine
- Gradient Boosting
- XGBoost
- Neural Network

Models will be evaluated using appropriate metrics rather than relying only on accuracy.

Planned Intelligent Diagnosis

The final system is intended to produce results such as:

```
=====
                        FAULT DIAGNOSIS REPORT
=====

Fault Status      : FAULT DETECTED
Fault Type        : SLG
Affected Phase    : A
Faulted Bus       : 13

Fault Current     : 4.82 kA
Fault Impedance   : 0.10 pu

ML Prediction     : SLG
Model Confidence  : 98.1 %

Severity          : HIGH

=====
```




Technology Stack

Programming Language

Python

Numerical Computing

- NumPy
- SciPy

Data Processing

- Pandas

Visualization

- Matplotlib
- Plotly

Machine Learning

- Scikit-learn
- XGBoost (*planned*)

Dashboard

- Streamlit

Testing

- Pytest

Development

- Visual Studio Code
 - Git
 - GitHub
-



Installation

Clone the repository:

```
git clone https://github.com/Sree2011/IEEE18-Intelligent-Fault-Analysis.git
cd IEEE18-Intelligent-Fault-Analysis
```

Create a virtual environment:

Windows

```
python -m venv .venv
.venv\Scripts\activate
```

Install dependencies:

```
pip install -r requirements.txt
```



Running Stage 1

Run the power-system analysis:

```
python -m power_system.main
```

The program will:

1. Load the 18-bus system
 2. Construct the Y-bus matrix
 3. Run Newton-Raphson load flow
 4. Check convergence
 5. Display bus voltages and angles
 6. Display calculated power injections
-



Running Tests

Run:

```
pytest
```

The test suite verifies the fundamental numerical components of the project.



Design Philosophy

This project is intentionally being developed **without hardware**.

The objective is to build a software environment where electrical-system behavior can be:

Mathematically Modeled



Numerically Simulated



Analyzed



Automatically Diagnosed



Visually Explained

The electrical-engineering calculations will be implemented explicitly wherever practical instead of treating a third-party power-system package as a black box.

External libraries may later be used for **independent validation and benchmarking**.



Validation Strategy

The project will be validated progressively.

Electrical validation

Compare calculated quantities against:

- Analytical fault equations
- Independent implementations
- Known benchmark/reference results
- Cross-checks using established Python power-system tools where appropriate

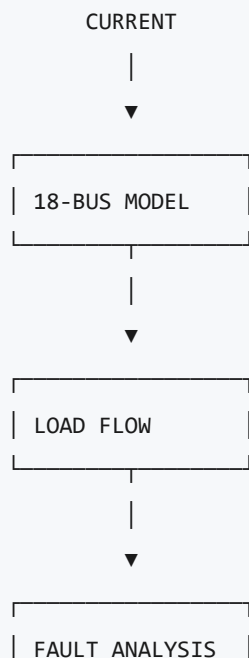
Machine-learning validation

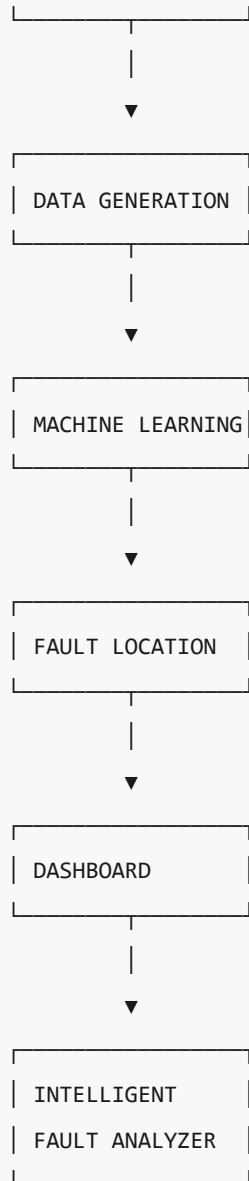
Evaluate:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Cross-validation performance

The objective is not simply to obtain a high accuracy number, but to determine whether the model produces **physically meaningful and reliable fault diagnoses**.

Long-Term Roadmap





Current Scope

The current network data is the **18-bus system used in the original development project**. It should not be assumed to be an official IEEE benchmark case unless independently verified.

The project architecture is designed so that verified benchmark systems can be added later.

Author

Sree

Electrical Engineering / Power-System Analysis Project

License

This project is licensed under the **MIT License**.

See [LICENSE](#) for details.

Project Vision

From power-system equations to intelligent fault diagnosis — entirely in Python.